

Week 2 Day 4 Academic Foundations

핵심 근거

근거	Day 4 연결
Docker Compose docs	여러 컨테이너 application stack을 YAML 파일과 CLI lifecycle로 관리
Compose Specification	services, networks, volumes, configs, secrets 같은 application model
Compose networking docs	같은 project network에서 service name 기반 discovery 제공
Compose services reference	ports, environment, env_file, depends_on, healthcheck의 공식 동작
PostgreSQL official image docs	초기화 environment, data directory, init script 실행 기준
Twelve-Factor App config principle	설정을 code/image에 굳히지 않고 환경별로 분리

시스템 관점

Compose는 단일 container 실행 명령의 나열을 application model로 묶는다. Day 3의 `docker run -p, -e, -v, --network` 옵션은 Day 4에서 service 정의의 일부가 된다. 이 전환의 핵심은 편의성이 아니라 재현성이다. 실행 조건이 shell history에 흩어져 있으면 다음 사람이 같은 결과를 만들기 어렵다.

Compose project는 service, network, volume을 하나의 이름공간으로 묶는다. `docker compose up`은 정의된 service container를 만들고, 필요한 network와 volume을 준비한다. 같은 network 안의 service는 container IP를 외우는 대신 service name으로 접근한다. 이 특성은 Week 3의 MSA와 Kubernetes Service 이해로 이어진다.

`depends_on`은 시작 순서를 도울 수 있지만 application readiness 전체를 보장하지 않는다. DB process가 시작됐다는 것과 query를 받을 준비가 됐다는 것은 다르다. Day 4에서는 healthcheck와 `pg_isready`로 이 차이를 관찰한다.

교육적 초점

Day 4의 난이도는 YAML 문법보다 실행 계약을 읽는 능력에 있다. 학생은 Compose 파일을 보고 "어떤 image가 실행되는가", "어떤 host port로 접근하는가", "어떤 설정이 외부에서 들어오는가", "데이터가 어디에 남는가", "장애 증거는 어디서 보는가"를 설명해야 한다.

평가 관점

수준	기대 행동
기억	services, ports, environment, volumes, networks 이름을 말한다
이해	Day 3 docker run 옵션과 Compose 항목을 mapping한다
적용	docker compose up, ps, logs, run, down을 실행한다
분석	missing env, wrong port, stale volume을 evidence로 분류한다
종합	README에 Compose 실행 계약과 cleanup 기준을 남긴다

공식 링크

- Docker Compose: <https://docs.docker.com/compose/>
- How Compose works: <https://docs.docker.com/compose/intro/compose-application-model/>
- Compose file reference: <https://docs.docker.com/compose/compose-file/>
- Compose services reference: <https://docs.docker.com/reference/compose-file/services/>
- Compose networks reference: <https://docs.docker.com/reference/compose-file/networks/>
- Networking in Compose: <https://docs.docker.com/compose/how-tos/networking/>
- PostgreSQL Docker Official Image: https://hub.docker.com/_/postgres
- Twelve-Factor App Config: <https://12factor.net/config>

Day 4 핵심 학습성과

Day 4의 학습성과는 "Compose를 안다"가 아니다. 관찰 가능한 행동으로 표현하면 다음과 같다.

학습성과	관찰 가능한 행동	산출 evidence
실행 조건 모델링	Day 3 docker run 옵션을 Compose 항목으로 변환	option mapping table
실행 전 검증	docker compose config로 YAML과 env를 확인	config output
multi-service 실행	web과 db를 project 단위로 실행	compose ps
service discovery 설명	container 내부에서 db service name을 사용	db-client output
readiness 판단	web HTTP와 DB health/query를 구분	curl, logs, query
장애 분류	missing env, wrong port, stale volume을 구분	RCA note
handoff 작성	run/check/cleanup/expected result 문서화	README section
전문 책임	secret과 data 삭제 위험을 명시	.env.example, cleanup warning

ABET-style Outcome Mapping

ABET Computing Student Outcomes 관점에서 Day 4는 다음 성과와 연결된다.

ABET-style outcome	Day 4 적용	평가 증거
문제 분석	긴 runtime command가 재현성에 취약한 이유 분석	Lesson 1 option risk note
computing-based solution 설계/구현/평가	compose.yaml로 web/db 실행 조건 구성	compose file, config, ps
커뮤니케이션	README handoff 작성	run/check/cleanup 문서
전문적 책임	secret 비노출, volume 삭제 위험 설명	.env.example, cleanup warning
협업 준비	다른 사람이 같은 환경을 재현 가능하게 기록	expected result와 blocker note

CS2023 Knowledge, Skill, Disposition

범주	Day 4 내용
Knowledge	Compose project, service, network, volume, environment, healthcheck 개념
Skill	config/up/ps/logs/run/down 명령을 실행하고 결과 해석
Disposition	증거 없는 완료 선언을 피하고, 재현 가능한 handoff를 우선

CS2023 관점에서 중요한 것은 지식과 기술만이 아니다. 학생이 secret 값을 공개하지 않고, data 삭제 명령을 위험도별로 구분하며, 실패 기록을 남기는 태도가 같이 평가되어야 한다.

NIST NICE-style Task/Knowledge/Skill

구분	Day 4 적용
Task	local multi-container environment를 구성하고 실행 상태를 검증
Knowledge	container network, runtime configuration, persistent storage, service health
Skill	logs/status/config evidence를 수집하고 failure domain을 분류
Professional behavior	credential exposure와 destructive cleanup을 피함

NIST NICE의 보안 직무 프레임은 Docker Compose 수업에도 적용할 수 있다. Compose 실습은 cloud 계정이나 production secret을 다루지 않더라도, configuration과 credential hygiene을 배우는 안전한 연습장이다.

Bloom's Taxonomy 적용

Bloom 단계	Day 4 질문
Remember	Compose의 services, networks, volumes 항목 이름을 말한다
Understand	host port와 container port, host DNS와 service DNS를 구분한다
Apply	docker compose up -d로 web/db를 실행한다
Analyze	실패를 config/start/runtime/cleanup 단계로 분류한다
Evaluate	down과 down -v 중 어떤 cleanup이 적절한지 판단한다
Create	개인 프로젝트용 Compose handoff section을 작성한다

SRE/DevOps 실무 기준

Day 4의 Compose 수업은 SRE/DevOps 관점에서 다음 기준을 포함해야 한다.

기준	Day 4 구현
Operational readiness	start/check/stop/cleanup 절차를 모두 포함
Reproducibility	path, port, env, image tag를 문서화
Observability	ps, logs, curl, query evidence 사용
Incident learning	failure drill과 RCA 템플릿 포함
Risk classification	secret, volume, port, env 위험을 severity로 분류
Handoff documentation	README 예시와 evidence checklist 포함

공식 문서와 수업 활동 연결

공식 링크는 단순 참고 목록이 아니라 활동의 근거로 사용한다.

공식 문서	수업에서 확인할 것
Docker Compose overview	Compose가 multi-container application을 정의하고 실행하는 도구임
Compose application model	service, network, volume, project 모델
Compose services reference	ports, environment, depends_on, healthcheck 동작
Compose networking	service name으로 container 간 통신하는 방식
PostgreSQL official image	POSTGRES_PASSWORD, init script, data directory 동작
Twelve-Factor App Config	config를 code/image와 분리하는 원칙

실무 판단 원칙

Day 4에서 학생에게 반복해서 강조할 판단 원칙은 다음과 같다.

1. 실행 전에는 `docker compose config`를 본다. 2. 실행 후에는 `docker compose ps`만으로 끝내지 않는다. 3. web은 HTTP status와 body marker로 확인한다. 4. DB는 healthcheck, log, query로 확인한다. 5. service 간 연결은 container IP가 아니라 service name을 기록한다. 6. `.env.example`에는 실제 secret을 넣지 않는다. 7. `down`과 `down -v`는 같은 cleanup이 아니다. 8. 실패는 숨기지 않고 RCA 형식으로 남긴다.

필수 오해 점검

오해	바로잡는 설명
Compose가 Dockerfile을 대체한다	Dockerfile은 build, Compose는 runtime orchestration
localhost는 어디서나 host를 의미한다	실행 위치에 따라 의미가 달라진다
depends_on이면 DB 준비가 완전히 보장된다	readiness와 app retry는 별도다
.env는 secret manager다	로컬 편의 파일이며 노출 위험이 있다
down -v는 일반 cleanup이다	named volume data 삭제가 포함될 수 있다
container IP를 기록하면 정확하다	IP는 변할 수 있으므로 service name을 기록한다

평가 루브릭

항목	0점	1점	2점
Compose 구조	설명 못함	services만 설명	services/networks/volumes 구분
실행	실행 못함	일부 실행	web/db project 실행
검증	주장만 있음	ps만 있음	HTTP와 DB evidence 있음
장애 분석	없음	증상 기록	원인 후보, fix, recheck 있음
보안	secret 노출	주의 문구만 있음	.env.example과 비노출 기준
cleanup	없음	down만 있음	down/down -v 위험 구분
handoff	없음	명령만 있음	expected result와 failure note 포함

강한 evidence 예시

```
## Day 4 Evidence
- Config: `docker compose config` succeeded.
- Web: `0.0.0.0:18084->80/tcp`, `HTTP/1.1 200 OK`, `compose-site-v1`.
```

- DB: `db` service healthy, `current\_database=paperclip`.
- Network: `db-client` reached host `db`.
- Volume: `compose-app\_pgdata`; `down -v` deletes practice data.
- Failure: missing `POSTGRES\_PASSWORD` reproduced and fixed.
- Handoff: README includes setup, run, check, cleanup, expected output.

약한 evidence 예시

Compose 실행 완료.

이 한 줄은 평가할 수 없다. 어떤 service가 실행됐는지, 어떤 port로 확인했는지, DB가 정상인지, cleanup이 되었는지 알 수 없기 때문이다.

Day 5와 Week 3 전이

Day 5에서는 Dockerfile과 Compose를 통합한다. Day 4에서 만든 `compose.yaml`은 Week 2 최종 산출물의 핵심이다.

Week 3 MSA로 넘어가면 service 수가 늘고, service 간 API 호출이 중요해진다. Day 4의 service name, network boundary, runtime configuration, dependency readiness 경험은 MSA의 API host, database host, internal/external endpoint 구분으로 전이된다.

교육 운영 메모

Day 4는 학생별 환경 차이가 많이 드러나는 날이다. Windows WSL2, macOS Docker Desktop, Linux Docker Engine에 따라 daemon 연결, file sharing, port binding 증상이 다를 수 있다.

따라서 수업 운영에서는 다음 순서를 유지한다.

1. 전체 공통 개념 설명
2. 표준 실습 앱 기준 실행
3. 실패 학생은 config/daemon/port/env/volume으로 분류
4. 개인 프로젝트 적용은 보충 또는 도전 과제로 이동
5. 종료 전 README handoff를 최소 1개 완성

전문 책임 문장

학생 README 또는 회고에 다음 수준의 문장이 포함되면 좋다.

이 Compose 구성은 로컬 실습용입니다. `.env`에는 실제 운영 secret을 넣지 않으며, 공개 repository에는 `.env.example`만 포함합니다. `docker compose down -v`는 PostgreSQL named volume을 삭제하므로 실습 초기화 외에는 사용하지 않습니다.

이 문장은 기술 실행과 전문적 책임을 함께 보여준다.

Day 4 완료 판정

Day 4는 다음 조건을 만족할 때 완료로 본다.

완료 조건	증거
학술 정렬	ABET, CS2023, NIST NICE, Bloom 기준이 lesson 활동과 연결됨
실무 정렬	SRE/DevOps evidence, RCA, handoff, risk 기준 포함
실행 가능성	compose.yaml, .env.example, web/db lab files 존재
검증 가능성	config, ps, HTTP, DB query, cleanup 명령이 문서화됨
책임성	secret 비노출과 volume 삭제 위험이 명시됨

이 기준은 수업 자료가 단순 요약문이 아니라 평가 가능한 실습 교안임을 확인하기 위한 최소선이다.